PHY 112L
Compressing a 2-D Ideal Gas

Michael Mulhearn

October 20, 2026

1 Ideal Gas in Two Dimensions

For this assignment, we will build a molecular dynamics model for an ideal gas in two dimensions. The ideal gas law is:

$$P\,V = N\,k\,T \tag{1}$$

where P is the pressure, V is the volume occupied by the gas, N is the number of molecules in the gas, k is Boltzmann's constant, and T is the temperature. The Equipartition theorem, applied to the ideal gas, leads to a total energy of the gas

$$U = \frac{f}{2}NkT \tag{2}$$

where f is the number of quadratic degrees of freedom: 3 for an ideal-gas in 3-D, and 2 for an ideal-gas in 2-D. The molecules in an ideal gas have only kinetic energy, and so:

$$U = N\,\langle K\rangle$$

where $\langle K\rangle$ is the average kinetic energy across all molecules, and so:

$$\langle K\rangle = \frac{f}{2}kT \tag{3}$$

In two dimensions (2-D), the ideal gas law becomes:

$$O\,A = N\,k\,T \tag{4}$$

where A is the area occupied by the gas, and O is the 2-D analog to pressure: a force per unit length. We will still call O the pressure of the 2-D gas. If one knows the pressure O as a function of the area A, then one can calculate the work done on the gas compressing (or expanding) from area A_1 to A_2 as:

$$W = -\int_{A_1}^{A_2} O(A)\;dA \tag{5}$$

The temperature is related to average kinetic energy across all molecules as (setting $f = 2$ in the above):

$$\langle K\rangle = kT. \tag{6}$$

The probability of finding a particular molecule with kinetic energy:

$$K = \frac{1}{2}mv^2$$

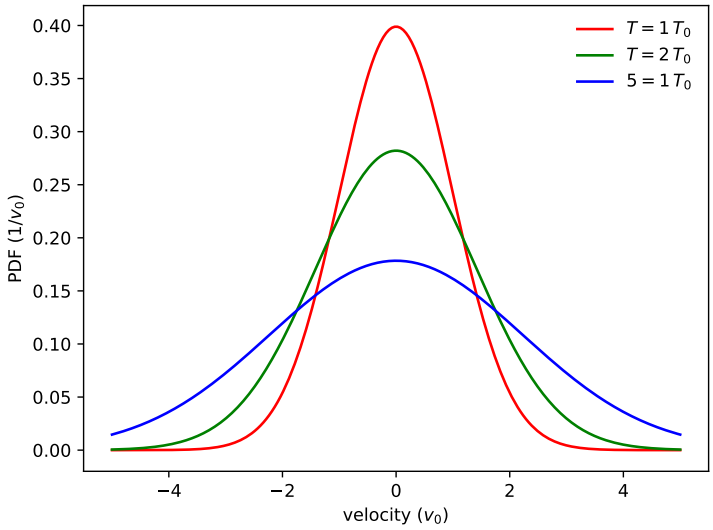


Figure 1: The PDF for each component of velocity of an ideal gas is a Gaussian distribution. The corresponding PDF for the speed is the Maxwell-Boltzmann distribution.

is proportional to the Boltzmann factor for the total energy E of the single molecule:

$$f(v)\,dv \;\propto\; \exp\left(-\frac{E}{kT}\right)dv \;=\; \exp\left(-\frac{mv^2}{2kT}\right)dv$$

because for an ideal gas $E = K$. Properly normalized this is the Maxwell-Boltzmann distribution. Assuming that the velocity components v_x and v_y are independent random variables, we conclude that:

$$f(v_x)\,dv_x = \left(\frac{m}{2\pi kT}\right)^{1/2} \exp\left(-\frac{mv_x^2}{2kT}\right)\,dv_x. \tag{7}$$

with a similar equation for dv_y. This PDF is illustrated in Fig. 1.

2 System of Units

When doing numerical work, it is important to use a well designed system of units. Computer programs have limited numerical precision, so we should never see numbers like

$$k = 1.38 \times 10^{-23} J/K$$

appear in our programs. Instead, the results of our calculations should be arranged to be near one. A good system of units removes tedious calculations from our code, making it more elegant and easy to follow. It makes it easy to spot bugs driving numbers very small or large. It also makes our numerical results more universal. For example, the results shown in Fig. 1 are equally valid for $T_0 = 1$ K and $T_0 = 1000$ C. There is no need to rerun the numerical calculation, which can be costly in both time and money. Just as accomplished chefs always prepare their *mise en place* before they begin cooking, we shall always prepare our system of units before programming.

We assume that our gas molecules have mass m and our gas has a relevant length scale a and temperature scale T_0. We take these as our units for reporting quantities of those types. Using

these units and the equations relevant to our model, we define other units. Motivated by the Maxwell-Boltzmann distribution, the velocity unit v_0 is defined by:

$$v_0^2 \equiv \frac{kT_0}{m}$$

and the unit for time is:

$$\tau \equiv a/v_0$$

Motivated by the ideal gas law, we can set the unit of pressure to:

$$O_0 \equiv \frac{NkT_0}{a^2}$$

and the unit of total energy:

$$U_0 = NkT_0$$

For analytic work, we keep these factors in place. But when using an analytic result within our computer programs, we set all of these quantities to 1:

$$m = a = T_0 = v_0 = \tau = O_0 = U_0 = 1$$

and so we should never see m or O_0 appear in our program as variables.

The ideal gas law:

$$OA = NkT$$

becomes, in our system of units:

$$\frac{O}{O_0} = \left(\frac{T}{T_0}\right)\left(\frac{a^2}{A}\right) \tag{8}$$

and within our program as:

```
def O(*, T, A):
    return T / A
```

The PDF for the x component of velocity becomes:

$$f(v_x/v_0)\, dv_x/v_0 = \left(\frac{T_0}{2\pi T}\right)^{1/2} \exp\left(-\frac{v_x^2}{2v_0^2} \cdot \frac{T_0}{T}\right)\, dv_x. \tag{9}$$

For analytic work we would say that the sigma of the Gaussian is:

$$\sigma = v_0\sqrt{T/T_0}$$

but in our program we would simply have:

```
sig = T**0.5
```

The temperature of an ideal gas is still calculated through the average kinetic energy:

$$kT = \langle K \rangle$$

which in our system of units becomes:

$$\frac{T}{T_0} = \frac{\langle v_x^2/v_0^2\rangle + \langle v_y^2/v_0^2\rangle}{2}$$

and could be calculated as:

```
def temp(*, vx, vy):
    return 0.5*(np.mean(vx**2) + np.mean(vy**2))
```

The `*` makes the arguments *keyword-only*: you call this function as `temp(vx=vx, vy=vy)` (Section 3).

3 Keyword-Only Arguments

Python lets you define a function so that every argument must be named when you call it: put a `*` first in the list of parameters. For example, the collision function of this assignment is defined as

```
def collide(*, ax, ay, bx, by, theta):
```

and called as

```
collide(ax=vx[a], ay=vy[a], bx=vx[b], by=vy[b], theta=theta)
```

The order of the named arguments does not matter, and a call such as `collide(1, 0, -1, 0, 0)` stops with an error (`TypeError`).

Most of our functions take several arguments of the same kind, numbers or arrays of numbers. If two of them are accidently swapped in a call, e.g. v_x and v_y, or the height of the piston and its velocity, Python cannot notice, and the program runs and gives the wrong physics. These are among the hardest bugs to find, and a common mistake. With keyword-only arguments every call says which value is which, and a missing or misspelled name is an error right away, at the line where it happens.

Course policy: every required function in this course is defined with keyword-only arguments, with exactly the names given in the assignment. We recommend the same for your own functions whenever they take more than one or two arguments.

4 Isothermal Compression

During an isothermal process the temperature is held constant and the process occurs quasistatically, so that the pressure is well-defined at each point:

$$PV = NkT = \text{const}$$

When a gas initially at pressure P_1 and volume V_1 is compressed to V_2, the pressure at each volume V is given by:

$$P(V) = \frac{P_1 V_1}{V}$$

and the work done on the gas is:

$$W = -\int_{V_1}^{V_2} P(V)\, dV \;=\; -P_1 V_1 \int_{V_1}^{V_2} \frac{dV}{V} \;=\; P_1\, V_1\, \ln(V_1/V_2)$$

Consider the 2-D gas column shown in Fig. 2. When the column is at height h the area of the gas is $A = ah$. In the isothermal case, the temperature remains at $T = T_1$. So the ideal gas law (as written in Equation 8) becomes:

$$\frac{O}{O_0} = \left(\frac{T_1}{T_0}\right)\left(\frac{a}{h}\right) \tag{10}$$

If the piston is initially at $y = h_1$ and we compress it until $y = h_2$, we have, in our system of units:

$$\frac{W}{U_0} = \frac{T}{T_0}\, \ln\, \left(\frac{h_1}{h_2}\right) \tag{11}$$

and in our program we would calculate the work as

```
def work_iso(*, T, hi, hf):
    return T * np.log(hi/hf)
```

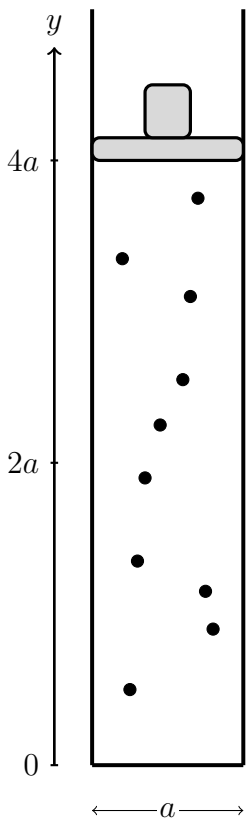


Figure 2: A 2-D gas cylinder of width a, containing an ideal gas of N particles. The plunger initially located at $y = 4a$ is used to compress the gas.

5 Adiabatic Compression (Not Needed Until HW 2)

During an adiabatic process, no thermal energy enters or leaves the gas, so

$$Q = 0$$

The total energy can still change by work done on the gas, with:

$$\Delta U = Q + W = W$$

Adiabatic processes are generally more difficult to calculate than isothermal because many macroscopic state properties change simultaneously: U,T,P and V. The one simplification we have is that

$$dU = dW = -PdV$$

From:

$$U = \frac{f}{2}NkT$$

and so:

$$\frac{f}{2}NkdT = -PdV$$

dividing by the ideal gas we obtain:

$$\frac{f}{2}\frac{dT}{T} = -\frac{dV}{V}$$

Integrating both sides we obtain:

$$\begin{aligned}
\int \frac{f}{2}\frac{dT}{T} &= -\int \frac{dV}{V} \\
\frac{f}{2}\ln(T) &= -\ln(V) + C \\
\ln\left(T^{f/2}\,V\right) &= C \\
T^{f/2}\,V &= C \\
T\,V^{2/f} &= C
\end{aligned}$$

Using the ideal gas law to write T in terms of P and V we obtain:

$$P\,V^{(f+2)/f} = C$$

In the case of 2-D gas with initial pressure O_1 and area A_1 the pressure O and area A during adiabatic compression or expansion is given by:

$$OA^2 = O_1 A_1^2$$

The work done compressing the gas from A_1 to A_2 is:

$$\begin{aligned}
W &= -\int_{A_1}^{A_2} O(A)\,dA \\
&= -O_1 A_1^2 \int_{A_1}^{A_2} \frac{dA}{A^2} \\
&= O_1 A_1^2 \;\frac{1}{A}\Big|_{A_1}^{A_2} \\
&= O_1 A_1 \left(\frac{A_1}{A_2} - 1\right) \\
&= NkT_1 \left(\frac{A_1}{A_2} - 1\right)
\end{aligned}$$

For the case of the gas cylinder shown in Fig. 2, with height of the column initially at h_1, the pressure during adiabatic compression is given by:

$$\frac{O}{O_0} = \left(\frac{T_1}{T_0}\right)\left(\frac{ah_1}{h^2}\right) \tag{12}$$

and the work done compressing the gas until the height is h_2 is:

$$\frac{W}{U_0} = \frac{T_1}{T_0}\left(\frac{A_1}{A_2} - 1\right) \tag{13}$$

6 Collision Model

At the heart of your numerical simulation is the collision model. It is the collisions of molecules that will allow your simulated gas to reach thermal equilibrium. We will use the simple elastic collision

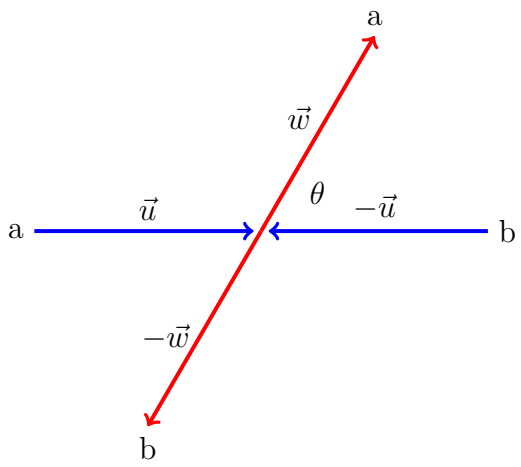


Figure 3: The collision model in the center-of-mass: incoming molecule a with velocity $\vec{u}$ collides with the incoming particle b of identical mass with velocity $-\vec{u}$. Particle a is scattered by angle θ and leaves with velocity $\vec{w}$, while particle b leaves with velocity $-\vec{w}$. The magnitude of the final and initial velocities are the same: $|\vec{u}| = |\vec{w}|$.

of identical mass particles, as illustrated in Fig. 3, as our collision model. We consider particles a and b with velocities $\vec{v_a}$ and $\vec{v_b}$ in the lab frame. The velocity of particle a in the CMS frame before the collision is

$$\vec{u} = \frac{\vec{v_a} - \vec{v_b}}{2}.$$

and the velocity of particle b is $-\vec{u}$.

The collision rotates the velocity of particle a by the scattering angle θ so that the velocity $\vec{w}$ after the collision is

$$\begin{pmatrix} w_x \\ w_y \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} u_x \\ u_y \end{pmatrix}$$

The velocity of particle b after scattering is $-\vec{w}$.

In the lab frame, the velocity of molecule a changes by an amount:

$$\Delta\vec{v_a} = \vec{w} - \vec{u}$$

and the velocity of molecule b changes by an amount:

$$\Delta\vec{v_b} = (-\vec{w}) - (-\vec{u}) = -\Delta\vec{v_a}$$

Our Python implementation for the collision will be computed in terms of the components of the velocity vectors of molecule a and molecule b:

$$\begin{aligned} \vec{v_a} &= \begin{pmatrix} a_x \\ a_y \end{pmatrix} \\ \vec{v_b} &= \begin{pmatrix} b_x \\ b_y \end{pmatrix} \end{aligned}$$

We'll use the Python variable names `ax`, `ay`, `bx`, and `by` to refer to a_x, a_y, b_x, and b_y.

You will implement the collision model as a Python function:

```
def collide(*, ax, ay, bx, by, theta):
    # your code here
    return ax, ay, bx, by # updated values
```

which takes the lab frame velocities of particle a (ax,ay) and b (bx, by) as keyword-only arguments (Section 3) and returns the lab frame velocities after scattering by an angle theta. **Please stick to these variable names and function definition in your own program**. It will help us help you debug your code.

To implement the algorithm, first calculate the x and y component of $\vec{u}$ as:

$$\begin{aligned} u_x &\equiv \frac{a_x - b_x}{2} \\ u_y &\equiv \frac{a_y - b_y}{2} \end{aligned}$$

Then compute the x and y component to the change in velocity of particle a and particle b:

$$\begin{aligned} \Delta a_x &= (\cos\theta - 1)\, u_x - \sin\theta\, u_y \\ \Delta a_y &= (\cos\theta - 1)\, u_y + \sin\theta\, u_x \end{aligned}$$

Finally, update the x and y components of the particle velocities to their value after the collision:

$$\begin{aligned} a_x &\rightarrow a_x + \Delta a_x \\ a_y &\rightarrow a_y + \Delta a_y \\ b_x &\rightarrow b_x - \Delta a_x \\ b_y &\rightarrow b_y - \Delta a_y \end{aligned}$$

Some test values with known answers are provided in the homework section. Make sure your collision model is rock solid before trying anything else!

7 Thermal Equilibrium of a Molecular Dynamics Model

Our model will consist of $N = 10000$ molecules constrained to a 2-D cylinder of width a and initial height $H = 4a$. We will track both components of each particle's velocity v_x and v_y, but we will only track each particle's y position. Nothing we are doing will depend on the x position, so we do not track it: we can imagine each particle bouncing off the side walls as needed to stay in position. In python, this would be coded as:

```
NGAS=10000
H  = 4
vx = np.random.uniform(size=NGAS, low=-3, high=3)
vy = np.random.uniform(size=NGAS, low=-3, high=3)
y  = np.random.uniform(size=NGAS, low= 0, high=H)
```

After calling this code, the distribution in each variable can be displayed as a histogram, as in the following snippet:

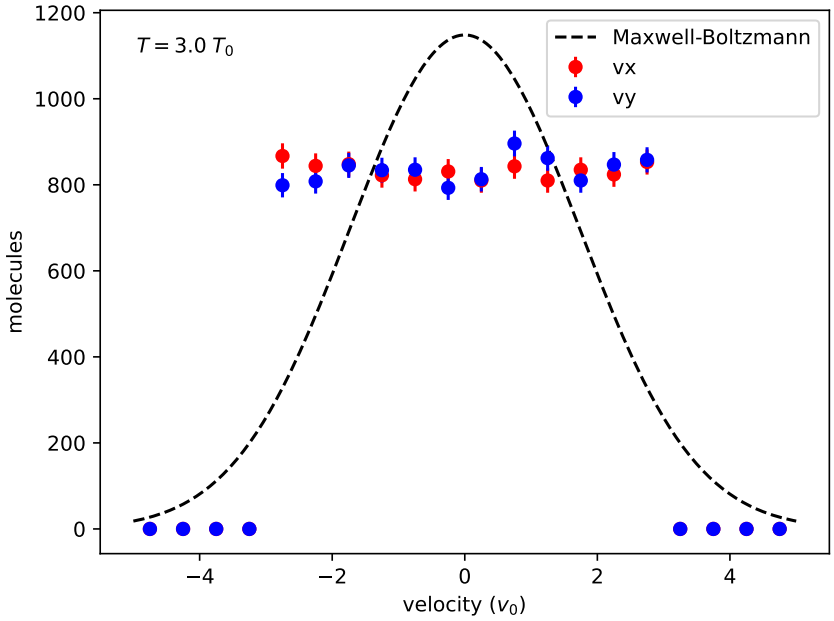


Figure 4: Velocity distribution immediately after initialization.

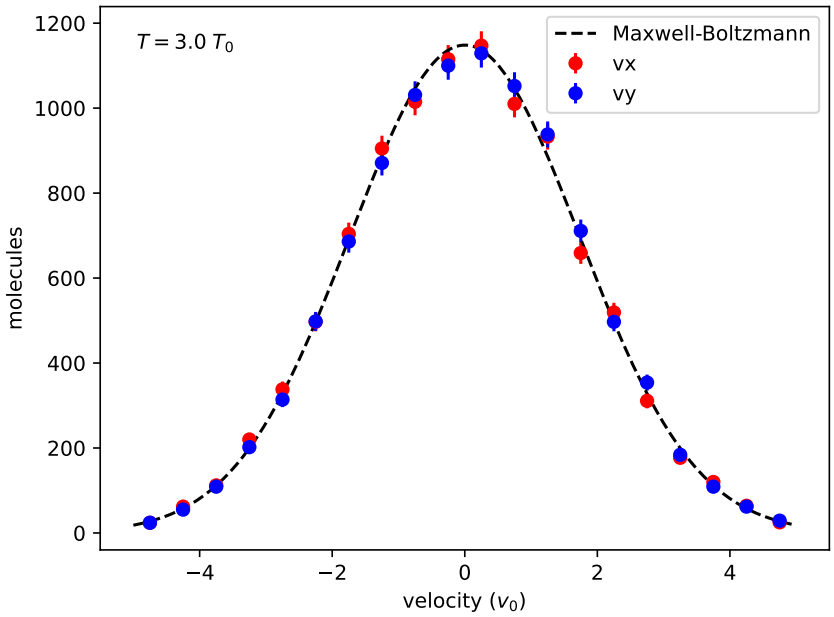


Figure 5: Thermal equilibrium.

```
hvx,bins = np.histogram(vx,bins=20,range=(-5,5))
cbins = (bins[1:]+bins[:-1])/2
plt.errorbar(cbins, hvx,fmt="ro", yerr=np.sqrt(hvx),label="vx")
```

Immediately after initialization, as shown in Fig. 4 the gas is not in thermal equilibrium, due to our non-physical initialization of the momenta. We will use our simple collision model to allow the gas to reach equilibrium. To keep things simple, we won't insist on physical proximity (the y values) in order to collide particles. We'll simply choose pairs at random, such as with:

```
a,b = np.random.choice(NGAS,size=2, replace=False)
```

and collide particles a and b by updating their v_x and v_y coordinates using the collision function and a randomly chosen value of θ. After 100k collisions, the system reaches thermal equilibrium, as shown in Fig. 5.

The temperature of the gas remains the same, however, as the collisions conserve energy. In order to set the temperature, we will let the system reach thermal equilibrium with a reservoir at the desired temperature. To simulate this process, we'll draw random values from the Maxwell-Boltzmann distribution of the velocity components:

```
Tr = 1
rx = np.random.normal(0,np.sqrt(Tr),size=n)
ry = np.random.normal(0,np.sqrt(Tr),size=n)
```

and collide these n reservoir particles with another n randomly chosen gas particles. Each reservoir particle should be used for one collision only and then discarded, to simulate an infinitely large reservoir.

8 Bouncing and Compression

Our numerical model includes the y-position of each molecules. While inside the container, a molecule's position can be updated for a time step Δt as:

$$y \to y + v_y \Delta t$$

This is fine, for as long as the particles remain within the container. However, particles with negative v_y will eventually be updated to have $y < 0$. These particles are bounced back into the container by the simple update:

$$y \to -y \tag{14}$$

and

$$v_y \to -v_y. \tag{15}$$

Meanwhile, particles moving upward will eventually be updated to have $y > h$, where h is the current location of the plunger. In this case, the particles are bounced back in by the update:

$$y \to 2h - y \tag{16}$$

If the piston is stationary, the velocities are updated as:

$$v_y \to -v_y. \tag{17}$$

However, we'll need to model a moving piston during compression.

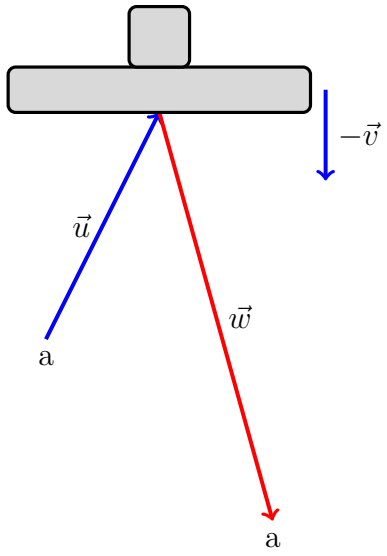


Figure 6: Gas particle a at velocity $\vec{u}$ collides with the plunger, travelling downward with velocity $-\vec{v}$, and recoils with new velocity $\vec{w}$.

During compression, molecules strike the moving plunger and recoil, as illustrated in Fig. 6. The particle a has initial velocity $\vec{u}$. After striking the plunger it recoils with new velocity $\vec{w}$. Your model will need to accurately simulate these collisions. The x-component of velocity is unchanged during the collision:

$$w_x = u_x$$

and we don't model the x-position of molecules. The y-component of the velocity after the collision with the plunger is given by:

$$w_y = -u_y + 2\,v_y \tag{18}$$

Note carefully that a piston moving down, as for compression, will have $v_y < 0$.

The move function

You will implement the motion of the molecules, including all bouncing, as a Python function:

```
def move(*, y, vy, h, vp, dt):
    # your code here
    return y, vy, dvy # updated values
```

which takes the arrays `y` and `vy` of positions and y-velocities of all molecules, the location `h` of the piston, the y-velocity `vp` of the piston (`vp = 0` for a stationary piston, `vp < 0` during compression), and the time step `dt`. It should:

- update every position by $y \to y + v_y \Delta t$,
- bounce molecules with $y < 0$ off the bottom ($y \to -y$, $v_y \to -v_y$) and molecules with $y > h$ off the piston ($y \to 2h - y$, $v_y \to -v_y + 2v_p$: Equation 18, with the piston velocity written as v_p), **repeating until every molecule is inside the container**, $0 \le y \le h$,
- return the updated arrays `y` and `vy`, and `dvy`: the total momentum (per unit mass) given to the piston, which is the sum of ($v_y^{\text{before}} - v_y^{\text{after}}$) over every bounce off the piston.

Keep `h` fixed during a single call; when the piston moves (Problem 6), update `h` between calls. **Please stick to this function name and definition.** A test value: a single molecule at $y = 1$ moving up with $v_y = 1$ in a container with a stationary piston at $h = 4$ crosses the container twice in $\Delta t = 10$:

```
y1, vy1, dvy1 = move(y=np.array([1.0]), vy=np.array([1.0]), h=4, vp=0, dt=10)
print(y1, vy1, dvy1)
```

should output `[3.]  [1.]  2.0`.

9 Checking Your Notebook: hw1check

This assignment comes with a checker, `hw1check.py`, available with the assignment on the course website. Put it in the same folder as your notebook. If you work on a cloud server (e.g. JupyterHub or Google Colab), upload `hw1check.py` next to your notebook there. Do not modify it. Use it as follows:

- **Import it** in the first cell of your notebook, along with everything else you need:

  ```
  # Setup
  import numpy as np
  import matplotlib.pyplot as plt
  import hw1check
  ```

- **Label your cells.** The first line of every other code cell is a comment naming the problem it belongs to: `# Problem 1`, `# Problem 2`, ..., `# Problem 6`.

- **Required functions.** Define `collide`, `temp` and `move` exactly as described in the text, with keyword-only arguments (Section 3), at the top level of your notebook (not inside another function), and use them in your solution.

- **Report each result** once, with `hw1check.report`, in the cell where you calculate it:

  ```
  hw1check.report("P2 T", T)
  ```

  The results to report are listed with each problem: `"P2 T"`, `"P3 T"`, `"P4 T"`, `"P5 O"`, `"P6 A"`, `"P6 O"` and `"P6 W"`.

- **Register each required plot** by calling `hw1check.figure` right after making it, in place of `plt.show()`:

  ```
  hw1check.figure("P2")
  ```

  The required plots are `"P2"`, `"P3"`, `"P4"` and `"P6"`.

- **Run the checker in the last cell:**

  ```
  import hw1check; hw1check.run()
  ```

  It tests your `collide` and `move` functions with the test values in this assignment, checks that every result and plot is there, tells you whether each result is correct to within the expected precision, and warns you about signs of cells run out of order. It ends either with "ready to submit" or with a list of what is still to do.

Before submitting, use Kernel → Restart & Run All, and check that the last cell says "ready to submit". If you cannot get there, add a text (markdown) cell at the end of your notebook explaining why. An honest result outside the expected precision, with your explanation, is better than one tuned to pass. Passing the checker does not guarantee credit: we run additional tests when grading.

10 Homework Problems

Please submit one PDF file (including output) and one python notebook file (.ipynb) of your completed assignment, after Restart & Run All, with `hw1check.run()` in the last cell. This assignment must be done in Python, in a Jupyter notebook. You will receive **zero credit if you do not use our system of units consistently!** When asked to calculate a quantity, e.g. temperature, answer in our units, e.g. T_0. Do not first calculate in SI units and then convert!

Problem 1: Implement the collision model described in the text. Test your code with some simple cases first, rotations by zero,quarter,half,and three quarters of a full turn:

```
tau = 2*np.pi # Using 2 pi is like saying twice half-way...
# lab frame is cms, incoming on x axis:
for theta in [0, tau/4, tau/2, 3*tau/4]:
    print(np.around(collide(ax=1, ay=0, bx=-1, by=0, theta=theta), 2) + 0)
```

which should have the output:

```
[ 1.  0. -1.  0.]
[ 0.  1.  0. -1.]
[-1.  0.  1.  0.]
[ 0. -1.  0.  1.]
```

And:

```
# lab frame is cms, incoming on y axis:
for theta in [0, tau/4, tau/2, 3*tau/4]:
    print(np.around(collide(ax=0, ay=1, bx=0, by=-1, theta=theta), 2) + 0)
```

which should have the output:

```
[ 0.  1.  0. -1.]
[-1.  0.  1.  0.]
[ 0. -1.  0.  1.]
[ 1.  0. -1.  0.]
```

Then test your collision function where the lab frame is not the CMS frame:

```
# boost along x axis, incoming on y axis:
for theta in [0, tau/4, tau/2, 3*tau/4]:
    print(np.around(collide(ax=1, ay=1, bx=1, by=-1, theta=theta), 2) + 0)
```

which should output:

```
[ 1.  1.  1. -1.]
[0. 0. 2. 0.]
[ 1. -1.  1.  1.]
[2. 0. 0. 0.]
```

Finally, test your collision function with randomly chosen values:

```
# test with random values:
tests = [(6.24, 1.78, 3.35, 5.98, 3.19), (4.07, 4.69, 1.61, 4.54, 2.46),
         (5.28, 2.99, 4.77, 5.22, 3.15), (2.84, 5.37, 5.47, 6.16, 1.59)]
for ax, ay, bx, by, theta in tests:
    print(np.around(collide(ax=ax, ay=ay, bx=bx, by=by, theta=theta), 2))
```

which should output:

```
[3.25 5.91 6.34 1.85]
[1.84 5.33 3.84 3.9 ]
[4.76 5.22 5.29 2.99]
[4.58 4.46 3.73 7.07]
```

Being able to develop good testing regimens is an essential programming skill.

Problem 2: Initialize the numerical model as described in the text, and reproduce Fig. 4. Calculate the temperature using your `temp` function. Report it as `"P2 T"` and register your plot as `"P2"`.

Problem 3: By applying your collision model to randomly chosen molecules 100,000 times, bring your model to thermal equilibrium and reproduce Fig. 5. Calculate the temperature. It should be unchanged from Problem 2. Report it as `"P3 T"` and register your plot as `"P3"`.

Problem 4: Bring your model into thermal equilibrium with a reservoir at $T = 1T_0$. Apply your collision model to a randomly chosen gas molecule and a randomly drawn reservoir particle, as described in the text, 100,000 times. Plot the resulting histogram, as in the previous problems. Calculate the temperature. It should match the reservoir! Report it as `"P4 T"` and register your plot as `"P4"`.

Problem 5: Simulate pressure with the gas at $T = 1T_0$ and the plunger **stationary** at $h = 4a$. Use your `move` function to update the y position of each molecule for a time interval $\Delta t = 10\tau$, bouncing at both $y = h$ and $y = 0$. The total Δv_y for particles bounced at the piston is `dvy`. In our system of units, the pressure can be calculated as:

$$\frac{O}{O_0} = \frac{1}{N}\frac{\Delta v_y}{v_0}\frac{\tau}{\Delta t}.$$

Compare your simulated pressure with your expectation from Equation 8. Report it as `"P5 O"`.
Hint: the time interval of 10τ was chosen to give you plenty of collisions for an accurate estimate of the pressure. However, it means that some particles will travel more than twice the length of the cylinder during this time. In order to get accurate results, you will have to keep bouncing **multiple times** until every particle is located within the volume. If your answer is close to the correct one but slightly low, this is almost certainly your bug. You have a small leak in your canister and therefore lower pressure!

Problem 6: Simulate isothermal compression of the gas from $h = 4a$ to $h = 2a$ in 20 timesteps of size $\Delta t = 10\tau$. Set the speed of the piston appropriately. Repeat the pressure calculation from the previous problem at each step with your `move` function, updating the location of the piston before each step, and accounting for its speed in the momentum transferred to bouncing molecules, as in Equation 18. Simulate an isothermal process by bringing the system into thermal equilibrium

with a reservoir at $T = 1T_0$ after each step. Plot your measured O versus A and compare to your expectation from Equation 10. Integrate your curve to get the work done, and compare to your expectation from Equation 11. Report the area $A = ah$ and the pressure O of each step as arrays `"P6 A"` and `"P6 O"`, and the work as `"P6 W"`. Register your plot as `"P6"`.

A Converting a Finished Notebook (This Year Only)

If you finished HW 1 before keyword-only arguments were introduced (Section 3), your `collide`, `temp` and `move` probably take positional arguments. `hw1check` accepts that with a warning, so you do not need to change your HW 1 submission. HW 2 reuses these functions and requires them to be keyword-only, so you will need to convert them before then.

You may do the conversion by hand, which takes a few minutes. **As an explicit exception to the AI policy in the syllabus, you may also use a generative AI tool for this conversion, and for nothing else**: it may add the `*` to the definitions of `collide`, `temp` and `move` and name the arguments in every call to them, and make no other change to your code. If you use AI:

- check every changed line yourself, then use Kernel → Restart & Run All and check that `hw1check` no longer warns;
- add a text (markdown) cell at the top of your notebook saying that you used AI for the keyword-only conversion, and which tool.

An example prompt (paste your code after it):

> Convert the Python functions `collide`, `temp` and `move` in the code below to keyword-only arguments, by adding `*` as the first parameter, for example `def collide(*, ax, ay, bx, by, theta)`. Then change every call to these three functions so that each argument is passed by name, for example `collide(ax=vx[a], ay=vy[a], bx=vx[b], by=vy[b], theta=theta)`. Do not change anything else: no other lines, names, numbers, comments or formatting, and do not fix or improve the code, even if you see a mistake. Afterwards, list every line you changed.